

A **matrix** is a two-dimensional data object made of m rows and n columns, therefore having total m x n values.

If most of the elements of the matrix have **0 value**, then it is called a sparse matrix.

Why to use Sparse Matrix instead of simple matrix ?

- **Storage:** There are lesser non-zero elements than zeros and thus lesser memory can be used to store only those elements.
- **Computing time:** Computing time can be saved by logically designing a data structure traversing only non-zero elements..

Example:

0 0 3 0 4

0 0 5 7 0

0 0 0 0 0

0 2 6 0 0

Representing a sparse matrix by a 2D array leads to wastage of lots of memory as zeroes in the matrix are of no use in most of the cases.

So, instead of storing zeroes with non-zero elements, we only store non-zero elements. This means storing non-zero elements with **triples- (Row, Column, value)**.

$$\begin{bmatrix} 1 & & & & \\ 5 & 3 & & & \\ 2 & 7 & -1 & & \\ 3 & 1 & 4 & 2 & \\ -9 & 2 & -8 & 1 & 7 \end{bmatrix}$$

Figure 3.34 Lower-triangular matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ & 3 & 6 & 7 & 8 \\ & & -1 & 9 & 1 \\ & & & 9 & 2 \\ & & & & 7 \end{bmatrix}$$

Figure 3.35 Upper-triangular matrix

$$\begin{bmatrix} 4 & 1 & & & \\ 5 & 1 & 2 & & \\ & 9 & 3 & 1 & \\ & & 4 & 2 & 2 \\ & & & 5 & 1 & 9 \\ & & & & 8 & 7 \end{bmatrix}$$

Figure 3.36 Tri-diagonal matrix

There are two types of sparse matrices. In the first type of sparse matrix, all elements above the main diagonal have a zero value. This type of sparse matrix is also called a (*lower*) *triangular matrix* because if you see it pictorially, all the elements with a non-zero value appear below the diagonal.

$$\begin{bmatrix} 1 & & & & \\ 5 & 3 & & & \\ 2 & 7 & -1 & & \\ 3 & 1 & 4 & 2 & \\ -9 & 2 & -8 & 1 & 7 \end{bmatrix}$$

Figure 3.34 Lower-triangular matrix

In a lower triangular matrix, $A_{i,j}=0$ where $i < j$.

An $n \times n$ lower-triangular matrix A has one non-zero element in the first row, two non-zero elements in the second row and likewise n non-zero elements in the nth row. Look at Fig. 3.34 which shows a lower-triangular matrix. To store a lower-triangular matrix efficiently in the memory, we can use a one-dimensional array which stores only non-zero elements. The mapping between a two-dimensional matrix and a one-dimensional array can be done in any one of the following ways:

(a) Row-wise mapping—Here the contents of array A[] will be {1, 5, 3, 2, 7, -1, 3, 1, 4, 2, -9, 2, -8, 1, 7}

(b) Column-wise mapping—Here the contents of array A[] will be {1, 5, 2, 3, -9, 3, 7, 1, 2, -1, 4, -8, 2, 1, 7}

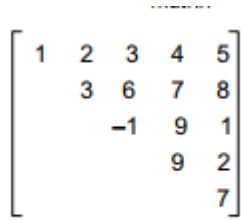


Figure 3.35 Upper-triangular matrix

In an upper-triangular matrix, $A_{i,j}=0$ where $i>j$.

An $n \times n$ upper-triangular matrix A has n non-zero elements in the first row, $n-1$ non-zero elements in the second row and likewise one non-zero element in the n th row. Look at Fig. 3.35 which shows an upper-triangular matrix.

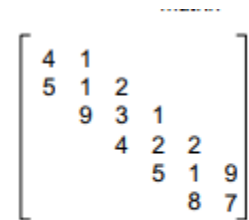


Figure 3.36 Tri-diagonal matrix

There is another variant of a sparse matrix, in which elements with a non-zero value can appear only on the diagonal or immediately above or below the diagonal. This type of matrix is also called a *tri-diagonal matrix*.

Hence, **In a tridiagonal matrix, $A_{i,j}=0$, where $|i - j| > 1$.**

In a tridiagonal matrix, if elements are present on (a) the main diagonal, it contains non-zero elements for $i=j$. In all, there will be n elements.

(b) below the main diagonal, it contains non-zero elements for $i=j+1$. In all, there will be $n-1$ elements.

(c) above the main diagonal, it contains non-zero elements for $i=j-1$. In all, there will be $n-1$ elements.

Figure 3.36 shows a tri-diagonal matrix. To store a tri-diagonal matrix efficiently in the memory, we can use a one-dimensional array that stores only non-zero elements. The mapping between a two-dimensional matrix and a one-dimensional array can be done in any one of the following ways:

(a) Row-wise mapping—Here the contents of array A[] will be

{4, 1, 5, 1, 2, 9, 3, 1, 4, 2, 2, 5, 1, 9, 8, 7}

(b) Column-wise mapping—Here the contents of array A[] will be

{4, 5, 1, 1, 9, 2, 3, 4, 1, 2, 5, 2, 1, 8, 9, 7}

(c) Diagonal-wise mapping—Here the contents of array A[] will be

{5, 9, 4, 5, 8, 4, 1, 3, 2, 1, 7, 1, 2, 1, 2, 9}

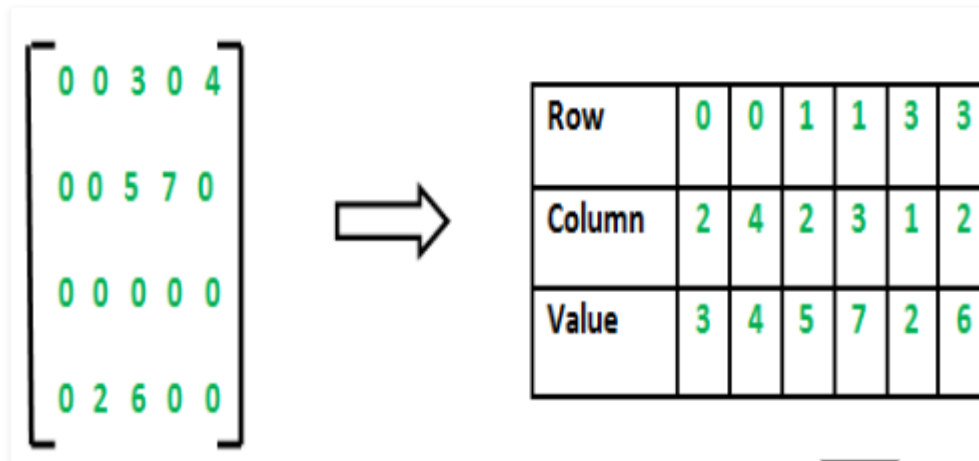
Sparse Matrix Representations can be done in many ways following are two common representations:

1. Array representation
2. Linked list representation

Method 1: Using Arrays:

2D array is used to represent a sparse matrix in which there are three rows named as

- **Row:** Index of row, where non-zero element is located
- **Column:** Index of column, where non-zero element is located
- **Value:** Value of the non zero element located at index – (row,column)



```
// C program for Sparse Matrix Representation
// using Array
#include<stdio.h>
```

```
int main()
{
    // Assume 4x5 sparse matrix
    int sparseMatrix[4][5] =
    {
        {0 , 0 , 3 , 0 , 4 },
        {0 , 0 , 5 , 7 , 0 },
        {0 , 0 , 0 , 0 , 0 },
        {0 , 2 , 6 , 0 , 0 }
    };

    int size = 0;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 5; j++)
            if (sparseMatrix[i][j] != 0)
                size++;

    // number of columns in compactMatrix (size) must be
    // equal to number of non - zero elements in
    // sparseMatrix
    int compactMatrix[3][size];

    // Making of new matrix
    int k = 0;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 5; j++)
            if (sparseMatrix[i][j] != 0)
            {
                compactMatrix[0][k] = i;
                compactMatrix[1][k] = j;
                compactMatrix[2][k] = sparseMatrix[i][j];
                k++;
            }
}
```

```
        }  
    for (int i=0; i<3; i++)  
    {  
        for (int j=0; j<size; j++)  
            printf("%d ", compactMatrix[i][j]);  
  
        printf("\n");  
    }  
    return 0;  
}
```

Output:

```
0 0 1 1 3 3  
2 4 2 3 1 2  
3 4 5 7 2 6
```

Process returned 0 (0x0) execution time : 0.602 s
Press any key to continue.